МЭДЭЭЛЛИЙН ТЕХНОЛОГИ,
КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

ӨГӨГДЛИЙН САН

БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Бие даалт № 10

Сэдэв: Түр хүснэгт ба хэрэглэгчийн тодорхойлсон функц (UDF)

Northwind өгөгдлийн сан, PostgreSQL 16

Гүйцэтгэсэн: Дугаржанцан Билгүүн

SISI дугаар: 23B1NUM1470

Хөтөлбөр: Компьютерын ухааны хөтөлбөр

Улаанбаатар хот

2026 он

Агуулга

Ажлын орчин 3

1 Нөөц дуусах дөхсөн бүтээгдэхүүний түр хүснэгт 4

2 ship_days() — хүргэлтийн хоногийг тооцох скаляр функц 6

3 Туршлагатай ажилтнуудын түр хүснэгт 8

4 freight_class() — тээврийн зардлын ангилал 10

5 Харилцагчийн идэвхийн түр хүснэгт 12

6 shipper_performance(p_year) — хүснэгт буцаадаг функц 15

7 pct_of() функц ба ангиллын орлогын бүтэц 17

8 customer_rfm() — функц доторх түр хүснэгт, сегментчлэл 19

9 restock_products() — өгөгдөл өөрчилдөг функц 22

10 abc_analysis() — Парето шинжилгээ ба гүйцэтгэлийн харьцуулалт 26

Ажлын орчин

Бүх даалгаврыг Windows 10 дээр суулгасан PostgreSQL 16.15 серверт, Northwind өгөгдлийн сан (91 харилцагч, 830 захиалга, 77 бүтээгдэхүүн, 2155 захиалгын мөр) дээр гүйцэтгэв. Баганын нэрс `snake_case` хэлбэртэй. Бүх код `23B1NUM1470_Дугаржанцан_Билгүүн_Биедаалт10.sql` файлд даалгаврын дарааллаар байрлах бөгөөд шинэ холболт дээр эхнээс нь дуустал алдаагүй ажилладаг (`psql -v ON_ERROR_STOP=1` горимоор шалгасан).

Хэрэглэх зарчим:

- Түр хүснэгт бүрийг `DROP TABLE IF EXISTS`-ээр эхэлж, хэрэглэж дууссаны дараа мөн `DROP TABLE IF EXISTS`-ээр устгасан. Функц доторх түр хүснэгтүүд `ON COMMIT DROP` тул дуудлагын transaction дуусахад өөрөө устна.
- Функц бүр `CREATE OR REPLACE FUNCTION` хэлбэртэй, бүх параметр `p_` угтвартай.
- Алдаа гаргах жишээ дуудлагуудыг SQL файлд `DO` блок дотор `EXCEPTION`-ээр барьж, алдааны мессежийг `NOTICE` болгон хэвлэсэн. Ингэснээр скрипт зогсолгүй дуусна. Тайлангийн зурагт эдгээр дуудлагыг шууд ажиллуулж, бодит `ERROR` мессежийг харуулав.
- Үр дүнгийн зургуудыг VS Code-ийн SQLTools өргөтгөлөөр (`Lab 3 Northwind` холболт) асуулга бүрийг ажиллуулж авсан. Зөвхөн CTE хувилбарын бүрэн төлөвлөгөө (Зураг 10.6) нь `psql` терминалын гаралт, учир нь SQLTools урт мөрийг тасалдаг.
- `freight`, `unit_price`, `discount` баганууд `real` төрөлтэй тул мөнгөн дүнг `::numeric` болгож тооцсон. `real`-ээс `numeric` рүү далд (implicit) хөрвүүлэлт байхгүй тул `NUMERIC` параметртэй функцэд `freight::numeric` гэж дамжуулна.

Даалгавар 1

Нөөц дуусах дөхсөн бүтээгдэхүүний түр хүснэгт

Ашигласан арга: түр хүснэгт (`CREATE TEMP TABLE ... AS`)

Дутагдалтай бүтээгдэхүүнийг `products` хүснэгтээс нэг удаа шүүж `low_stock` түр хүснэгтэд хадгалаад, түүн дээрээ хоёр тайланг ажиллуулсан. Ингэснээр шүүлтийн нөхцөлийг тайлан бүрт давтан бичих шаардлагагүй, түр хүснэгт зөвхөн энэ сешнд харагддаг тул байнгын схемд илүү объект үлдэхгүй. `shortage`-ийг `reorder_level - (units_in_stock + units_on_order)` гэж `SELECT` дотор тооцов.

SQL код

```sql
DROP TABLE IF EXISTS low_stock;

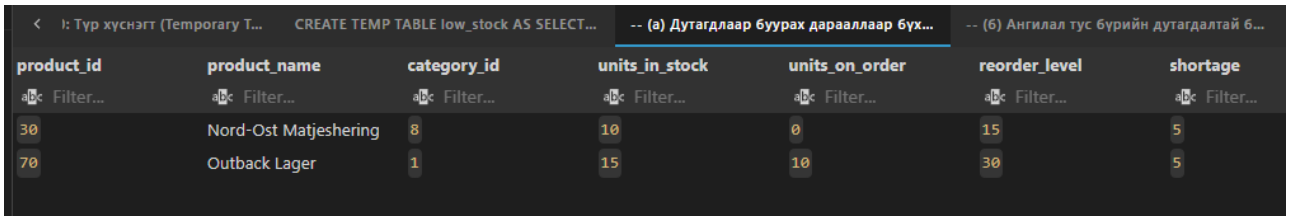
CREATE TEMP TABLE low_stock AS
SELECT
    product_id,
    product_name,
    category_id,
    units_in_stock,
    units_on_order,
    reorder_level,
    reorder_level - (units_in_stock + units_on_order) AS shortage
FROM products
WHERE units_in_stock + units_on_order < reorder_level
  AND discontinued = 0;

-- (а) Дутагдлаар буурах дараалаар бүх мөр
SELECT *
FROM low_stock
ORDER BY shortage DESC;

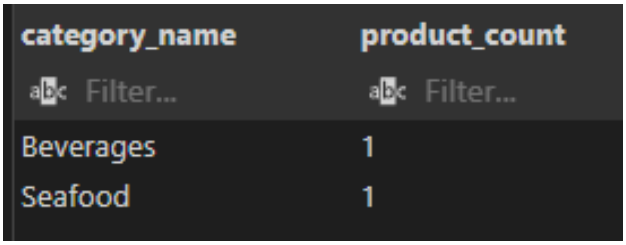
-- (б) Ангилал тус бүрийн дутагдалтай бүтээгдэхүүний тоо
SELECT c.category_name, COUNT(*) AS product_count
FROM low_stock AS ls
JOIN categories AS c ON c.category_id = ls.category_id
GROUP BY c.category_name
ORDER BY product_count DESC, c.category_name;

DROP TABLE IF EXISTS low_stock;
```

Үр дүн



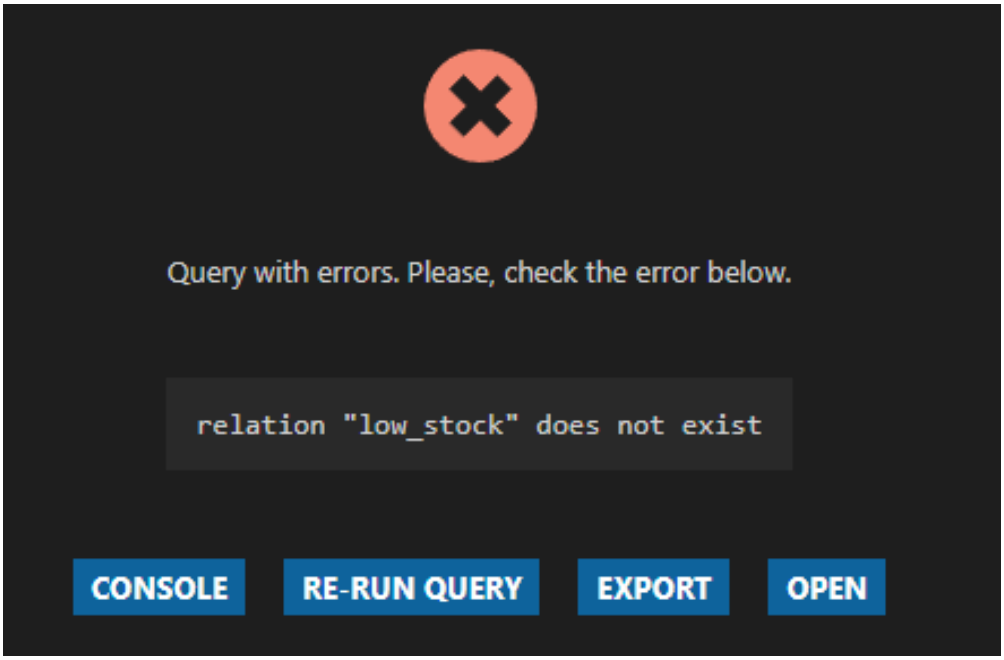
Зураг 1.1: Нөхцөлд тэнцсэн 2 бүтээгдэхүүн. Хоёулаа 5 ширхэгээр дутагдалтай.



Зураг 1.2: Beverages, Seafood ангилалд тус бүр 1 дутагдалтай бүтээгдэхүүн байна.

Асуулт: `SELECT * FROM low_stock;` ажиллуулбал юу болох вэ?

`DROP TABLE`-оос өмнө ижил сешнд ажиллуулбал хүснэгтийн 7 баганатай 2 мөрийг буцаана. Харин ажлын төгсгөлд хүснэгтийг устгасны дараа, эсвэл өөр холболтоос ажиллуулбал `ERROR: relation "low_stock" does not exist` алдаа гарна. Учир нь түр хүснэгт зөвхөн үүсгэсэн сешн дотроо, устгах эсвэл сешн хаагдах хүртэл л оршдог.



Зураг 1.3: `DROP` хийсний дараах `SELECT * FROM low_stock`.

Даалгавар 2

ship_days() — хүргэлтийн хоногийг тооцох скаляр функц

Ашигласан арга: скаляр SQL функц

Хүргэлтийн хоногийн тооцоо энэ даалгавар болон 6-р даалгаварт давтагддаг тул нэг функц болгосон. Тооцоо нь ганц илэрхийлэл тул `LANGUAGE sql` хамгийн энгийн сонголт. PostgreSQL-д `date - date` нь хоногийн тоог бүхэл тоогоор буцаадаг. `p_shipped_date` нь `NULL` бол хасах үйлдэл өөрөө `NULL` буцаах тул тусгай `IF` шалгалт шаардлагагүй.

Яагаад IMMUTABLE вэ?

Функцийн үр дүн зөвхөн хоёр оролтын огнооноос хамаарна. Хүснэгт уншдаггүй, `now()` зэрэг цаг эсвэл тохиргооноос хамаарах функц дууддаггүй тул ижил оролтод үргэлж ижил үр дүн өгнө. Иймд `IMMUTABLE` гэж зарласнаар planner функцийг асуулга руу inline хийх, тогтмол аргументтай дуудлагыг урьдчилан тооцох, мөн илэрхийллийн индекст (expression index) ашиглах боломжтой болно.

SQL код

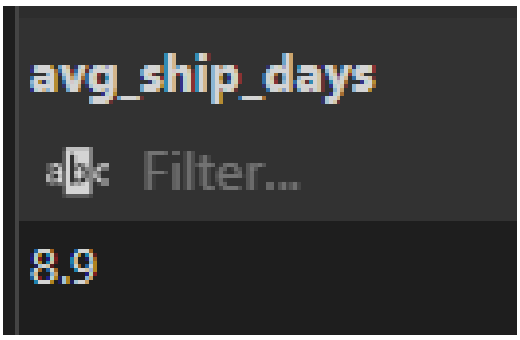
```sql
CREATE OR REPLACE FUNCTION ship_days(p_order_date DATE, p_shipped_date DATE)
RETURNS INT
LANGUAGE sql
IMMUTABLE
AS $$
    -- p_shipped_date нь NULL бол хасах үйлдэл өөрөө NULL буцаана
    SELECT p_shipped_date - p_order_date;
$$;

-- 1997 оны захиалгуудын дундаж хүргэлтийн хоног
SELECT ROUND(AVG(ship_days(order_date, shipped_date)), 1) AS avg_ship_days
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 1997;

-- 1997 оны хамгийн удаан хүргэгдсэн 5 захиалга
SELECT order_id, customer_id, order_date, shipped_date,
       ship_days(order_date, shipped_date) AS ship_days
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 1997
  AND shipped_date IS NOT NULL
ORDER BY ship_days DESC, order_id
LIMIT 5;

-- Хүргэгдээгүй захиалгын тоо
SELECT COUNT(*) AS not_shipped_orders
FROM orders
WHERE ship_days(order_date, shipped_date) IS NULL;
```

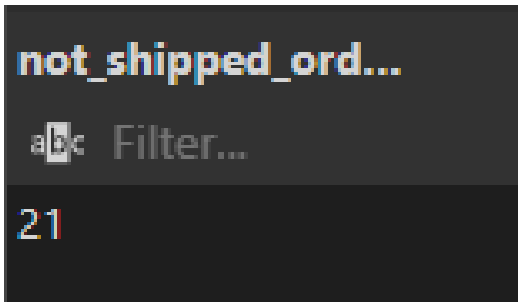
Үр дүн



Зураг 2.1: 1997 оны захиалгуудын дундаж хүргэлт 8.9 хоног.

order_id	customer_id	order_date	shipped_date	ship_days
Filter...	Filter...	Filter...	Filter...	Filter...
10660	HUNGC	1997-09-08	1997-10-15	37
10777	GOURL	1997-12-15	1998-01-21	37
10427	PICCO	1997-01-27	1997-03-03	35
10545	LAZYK	1997-05-22	1997-06-26	35
10593	LEHMS	1997-07-09	1997-08-13	35

Зураг 2.2: 1997 оны хамгийн удаан хүргэгдсэн 5 захиалга (35–37 хоног).



Зураг 2.3: `ship_days()` нь `NULL` буцаасан буюу хүргэгдээгүй 21 захиалга байна.

Даалгавар 3

Туршлагатай ажилтнуудын түр хүснэгт

Ашигласан арга: бүтцийг зарласан түр хүснэгт ба `INSERT ... SELECT`

`CREATE TEMP TABLE (...)` хэлбэрээр баганын төрөл болон `PRIMARY KEY`-г өөрөө зааж өгсөн тул давхардсан `employee_id` орох боломжгүй, төрөл нь тодорхой. Өгөгдлийг тусад нь `INSERT ... SELECT`-ээр оруулснаар хүснэгтийн бүтэц ба өгөгдөл бэлтгэх логик салж, код уншихад ойлгомжтой болсон. `full_name`-ийг `||` операторoор залгаж, `years_worked`-ийг `EXTRACT(YEAR FROM age(hire_date))` буюу өнөөдрийг хүртэлх бүтэн жилээр тооцов.

SQL код

```sql
DROP TABLE IF EXISTS senior_staff;

CREATE TEMP TABLE senior_staff (
    employee_id  INT PRIMARY KEY,
    full_name    TEXT,
    title        TEXT,
    hire_date    DATE,
    years_worked INT
);

INSERT INTO senior_staff (employee_id, full_name, title, hire_date, years_worked)
SELECT
    employee_id,
    first_name || ' ' || last_name,
    title,
    hire_date,
    EXTRACT(YEAR FROM age(hire_date))
FROM employees
WHERE hire_date < DATE '1994-01-01';

-- Ажилласан жилээр буурах дараалаар
SELECT *
FROM senior_staff
ORDER BY years_worked DESC, employee_id;

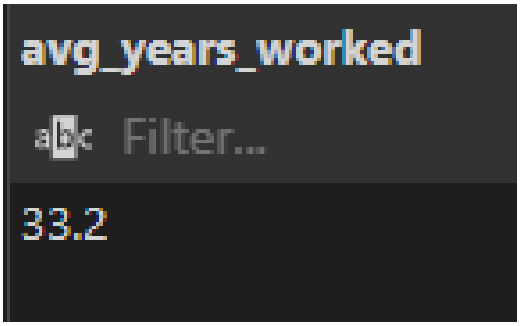
-- Ажилласан жилийн дундаж
SELECT ROUND(AVG(years_worked), 1) AS avg_years_worked
FROM senior_staff;

DROP TABLE IF EXISTS senior_staff;
```

Үр дүн

employee_id	full_name	title	hire_date	years_worked
1	Nancy Davolio	Sales Representative	1992-05-01	34
2	Andrew Fuller	Vice President, Sales	1992-08-14	34
3	Janet Leverling	Sales Representative	1992-04-01	34
4	Margaret Peacock	Sales Representative	1993-05-03	33
5	Steven Buchanan	Sales Manager	1993-10-17	32
6	Michael Suyama	Sales Representative	1993-10-17	32

Зураг 3.1: 1994-01-01-ээс өмнө ажилд орсон 6 ажилтан. `age()` нь өнөөдрийн огноог (2026-10-02) ашигладаг тул ажилласан жил ажиллуулсан өдрөөс хамаарна.



Зураг 3.2: Ажилласан жилийн дундаж 33.2 жил.

Даалгавар 4

freight_class() — тээврийн зардлын ангилал

Ашигласан арга: `CASE`-тэй скаляр SQL функц

Ангиллын дүрмийг нэг функцэд тодорхойлсноор `GROUP BY` болон `WHERE` хоёуланд ижил дүрэм хэрэглэгдэнэ. Хил хязгаар өөрчлөгдвөл зөвхөн функцийг засахад хангалттай. Функц зөвхөн оролтоосоо хамаардаг тул `IMMUTABLE`. `CASE` нь `NULL`-ийг эхэнд шалгаж `'Тодорхойгүй'` гэж буцаана, 50–150 (хоёр захаа оролцуулан) нь `'Дунд'`.

SQL код

```sql
CREATE OR REPLACE FUNCTION freight_class(p_freight NUMERIC)
RETURNS TEXT
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT CASE
        WHEN p_freight IS NULL THEN 'Тодорхойгүй'
        WHEN p_freight < 50    THEN 'Бага'
        WHEN p_freight <= 150  THEN 'Дунд'
        ELSE 'Өндөр'
    END;
$$;

-- freight багана real төрөлтэй тул NUMERIC параметрт ::numeric болгож дамжуулна.

-- 1997 оны захиалгууд ангилал тус бүрээр
SELECT
    freight_class(freight::numeric) AS freight_class,
    COUNT(*) AS orders_count,
    ROUND(SUM(freight::numeric), 2) AS total_freight,
    ROUND(AVG(freight::numeric), 2) AS avg_freight
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 1997
GROUP BY freight_class(freight::numeric)
ORDER BY avg_freight;

-- 1997 онд 'Өндөр' ангилалд орсон захиалга хамгийн олонтой 5 улс
SELECT ship_country, COUNT(*) AS high_freight_orders
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 1997
  AND freight_class(freight::numeric) = 'Өндөр'
GROUP BY ship_country
ORDER BY high_freight_orders DESC, ship_country
LIMIT 5;
```

Үр дүн

freight_class	orders_count	total_freight	avg_freight
Filter...	Filter...	Filter...	Filter...
Бага	227	4175.56	18.39
Дунд	122	10841.74	88.87
Өндөр	59	17451.47	295.79

Зураг 4.1: 1997 оны 408 захиалгын ангилал. `'Өндөр'` ангилал захиалгын 14%-ийг эзэлдэг ч нийт freight-ийн 54%-ийг бүрдүүлж байна. `freight` нь `NULL` захиалга байхгүй тул `'Тодорхойгүй'` мөр гараагүй.

ship_country	high_freight_ord...
Filter...	Filter...
Germany	12
USA	10
Austria	6
Canada	5
France	5

Зураг 4.2: 1997 онд `'Өндөр'` ангилалд орсон захиалга хамгийн олонтой 5 улс.

Даалгавар 5

Харилцагчийн идэвхийн түр хүснэгт

Ашигласан арга: түр хүснэгт ба `LEFT JOIN`

Харилцагч бүрийн захиалгын хураангуйг нэг удаа нэгтгэж `customer_activity`-д хадгалаад гурван тайланд дахин ашигласан. Ингэснээр `orders` хүснэгтийг гурван удаа `GROUP BY` хийх шаардлагагүй болсон. `LEFT JOIN` ба `COUNT(o.order_id)` хэрэглэсэн тул захиалгагүй харилцагч `orders_count = 0`-ээр орно (`COUNT(*)` бол 1 гэж тоолох байсан). `total_freight`-ийг `COALESCE`-ээр 0 болгосон.

SQL код

```
DROP TABLE IF EXISTS customer_activity;

-- LEFT JOIN тул захиалгагүй харилцагч ч orders_count = 0-ээр орно
CREATE TEMP TABLE customer_activity AS
SELECT
    c.customer_id,
    COUNT(o.order_id) AS orders_count,
    MIN(o.order_date) AS first_order,
    MAX(o.order_date) AS last_order,
    COALESCE(SUM(o.freight::numeric), 0) AS total_freight
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id;

-- (а) Хамгийн олон захиалга хийсэн 10 харилцагч
SELECT ca.customer_id, c.company_name, c.country, c.city, ca.orders_count
FROM customer_activity AS ca
JOIN customers AS c ON c.customer_id = ca.customer_id
ORDER BY ca.orders_count DESC, ca.customer_id
LIMIT 10;

-- (б) 1998 онд захиалга хийгээгүй боловч өмнө нь захиалж байсан харилцагчид
SELECT ca.customer_id, c.company_name, c.country, ca.orders_count, ca.last_order
FROM customer_activity AS ca
JOIN customers AS c ON c.customer_id = ca.customer_id
WHERE ca.orders_count > 0
  AND ca.last_order < DATE '1998-01-01'
ORDER BY ca.last_order, ca.customer_id;

-- (в) Улс тус бүрийн харилцагч, идэвхтэй харилцагч, нийт freight
SELECT
    c.country,
    COUNT(*) AS customers_count,
    COUNT(*) FILTER (WHERE ca.orders_count > 0) AS active_customers,
    ROUND(SUM(ca.total_freight), 2) AS total_freight
FROM customer_activity AS ca
JOIN customers AS c ON c.customer_id = ca.customer_id
GROUP BY c.country
ORDER BY total_freight DESC, c.country;

DROP TABLE IF EXISTS customer_activity;
```

Үр дүн

Түр хүснэгтэд бүх 91 харилцагч орсон бөгөөд тэдгээрийн 2 нь (FISSA, PARIS) огт захиалга хийгээгүй.

customer_id	company_name	country	city	orders_count
Filter...	Filter...	Filter...	Filter...	Filter...
SAVEA	Save-a-lot Markets	USA	Boise	31
ERNSH	Ernst Handel	Austria	Graz	30
QUICK	QUICK-Stop	Germany	Cunewalde	28
FOLKO	Folk och fä HB	Sweden	Bräcke	19
HUNGO	Hungry Owl All-Night Grocers	Ireland	Cork	19
BERGS	Berglunds snabbköp	Sweden	Luleå	18
HILAA	HILARION-Abastos	Venezuela	San Cristóbal	18
RATTC	Rattlesnake Canyon Grocery	USA	Albuquerque	18
BONAP	Bon app'	France	Marseille	17
FRANK	Frankenversand	Germany	München	15

Зураг 5.1: (а) Хамгийн олон захиалга хийсэн 10 харилцагч.

customer_id	company_name	country	orders_count	last_order
Filter...	Filter...	Filter...	Filter...	Filter...
CENTC	Centro comercial Moctezuma	Mexico	1	1996-07-18
LAZYK	Lazy K Kountry Store	USA	2	1997-05-22
HUNGC	Hungry Coyote Import Store	USA	5	1997-09-08
MEREP	Mère Paillarde	Canada	13	1997-10-30
FAMIA	Familia Arquibaldo	Brazil	7	1997-10-31
VINET	Vins et alcools Chevalier	France	5	1997-11-12
GROSR	GROSELLA-Restaurante	Venezuela	2	1997-12-18
FOLIG	Folies gourmandes	France	5	1997-12-22

Зураг 5.2: (б) 1998 онд захиалга хийгээгүй боловч өмнө нь захиалж байсан 8 харилцагч.

country	customers_count	active_customers	total_freight
Filter...	Filter...	Filter...	Filter...
USA	13	13	13771.29
Germany	11	11	11283.28
Austria	2	2	7391.50
Brazil	9	9	4880.19
France	11	10	4237.84
Sweden	2	2	3237.60
UK	7	7	2954.27
Ireland	1	1	2755.24
Venezuela	4	4	2735.18
Canada	3	3	2198.09
Denmark	2	2	1396.19
Switzerland	2	2	1368.53
Belgium	2	2	1280.14
Mexico	5	5	1122.78
Finland	2	2	910.89
Italy	3	3	864.44
Spain	5	4	861.89
Portugal	2	2	643.53
Argentina	3	3	598.58
Norway	1	1	275.50
Poland	1	1	175.74

Зураг 5.3: (в) Улс тус бүрийн үзүүлэлт (бүх 21 улс). Франц, Испанид идэвхгүй харилцагч тус бүр 1 байна.

Даалгавар 6

shipper_performance(p_year) — хүснэгт буцаадаг функц

Ашигласан арга: `RETURNS TABLE` хэлбэрийн PL/pgSQL функц

Оноос хамаарсан тайланг функц болгосноор дурын онд дахин ашиглаж, үр дүнг ердийн хүснэгт шиг `WHERE`, `ORDER BY`-аар боловсруулах боломжтой болсон. Ороltыг шалгаж `RAISE EXCEPTION` хийх шаардлагатай тул `plpgsql` сонгосон. `avg_ship_days`-ийг 2-р даалгаврын `ship_days()`-ээр, `late_pct`-ийг `NULLIF`-ээр 0-д хуваахаас хамгаалж тооцов. Функц зөвхөн өгөгдөл уншдаг ч хүснэгтийн агуулгаас хамаардаг тул `IMMUTABLE` биш, `STABLE` байна.

SQL код

```
CREATE OR REPLACE FUNCTION shipper_performance(p_year INT, p_min_orders INT DEFAULT 10)
RETURNS TABLE (
    shipper_name   TEXT,
    orders_count   BIGINT,
    avg_ship_days  NUMERIC,
    late_orders    BIGINT,
    late_pct       NUMERIC
)
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    IF p_year NOT BETWEEN 1996 AND 1998 THEN
        RAISE EXCEPTION 'p_year нь 1996-1998 хооронд байх ёстой (өгсөн утга: %)', p_year;
    END IF;

    RETURN QUERY
    SELECT
        s.company_name::TEXT,
        COUNT(*),
        ROUND(AVG(ship_days(o.order_date, o.shipped_date)), 1),
        COUNT(*) FILTER (WHERE o.shipped_date > o.required_date),
        ROUND(100.0 * COUNT(*) FILTER (WHERE o.shipped_date > o.required_date)
              / NULLIF(COUNT(*), 0), 1)
    FROM orders AS o
    JOIN shippers AS s ON s.shipper_id = o.ship_via
    WHERE EXTRACT(YEAR FROM o.order_date) = p_year
    GROUP BY s.company_name
    HAVING COUNT(*) >= p_min_orders;
END;
$$;

-- Байрлалаар дуудах
SELECT *
FROM shipper_performance(1997, 10)
ORDER BY late_pct DESC;

-- Нэрээр (=>) дуудах
SELECT *
FROM shipper_performance(p_year => 1998, p_min_orders => 80)
ORDER BY orders_count DESC;

-- Үр дүнг WHERE-ээр шүүх
SELECT shipper_name, avg_ship_days, late_pct
FROM shipper_performance(1996)
WHERE late_pct > 5;

-- Буруу он өгвөл алдаа гарна. Скрипт зогсохгүйн тулд DO блокоор барьж мессежийг харуулна.
DO $$
BEGIN
    PERFORM * FROM shipper_performance(2000);
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Алдаа: %', SQLERRM;
END;
```

```
$$;
```

Үр дүн

shipper_name	orders_count	avg_ship_days	late_orders	late_pct
United Package	153	10.2	11	7.2
Speedy Express	133	8.6	6	4.5
Federal Shipping	122	7.7	5	4.1

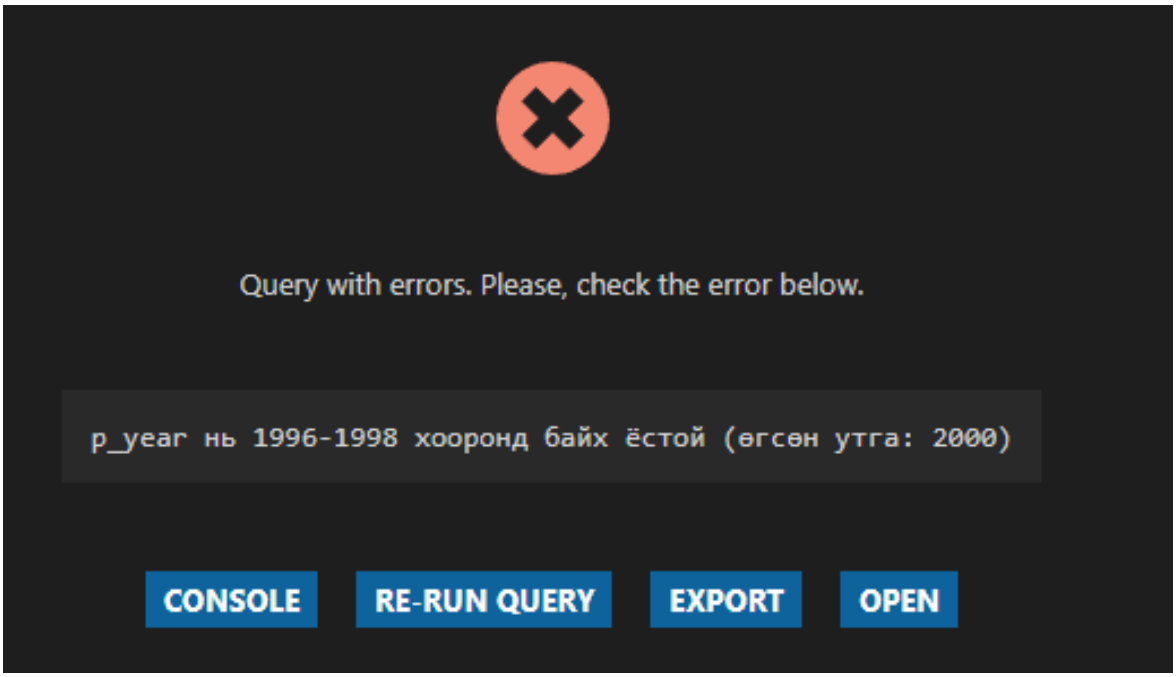
Зураг 6.1: Байрлалаар дуудах. 1997 онд United Package хамгийн их хоцролттой (7.2%).

shipper_name	orders_count	avg_ship_days	late_orders	late_pct
United Package	117	8.6	3	2.6

Зураг 6.2: Нэрээр (=>) дуудах. 1998 онд Speedy Express (78), Federal Shipping (75) 80-аас цөөн захиалгатай тул `HAVING`-ээр хасагдсан.

shipper_name	avg_ship_days	late_pct
Federal Shipping	8.2	5.2
Speedy Express	8.3	5.3

Зураг 6.3: Үр дүнг `WHERE`-ээр шүүх: 1996 онд хоцролт 5%-иас их тээвэрлэгчид.



Зураг 6.4: `SELECT * FROM shipper_performance(2000)` — 1996–1998-аас гадуурх он өгөхөд гарах алдаа.

Даалгавар 7

pct_of() функц ба ангиллын орлогын бүтэц

Ашигласан арга: туслах функц, түр хүснэгт, цонхны функц

Эзлэх хувийг 0 болон `NULL`-ээс хамгаалж тооцох томьёо энэ даалгавар болон 10-р даалгаварт хэрэглэгдэх тул `pct_of()` функц болгосон. 1997 оны ангиллын орлогыг (`unit_price * quantity * (1 - discount)`) нэг удаа нэгтгэж `category_revenue`-д хадгалснаар дараагийн хоёр асуулга 2155 мөрийг дахин нэгтгэхгүй, ердөө 8 мөр дээр ажиллана. Нийт орлогыг `SUM(revenue) OVER ()`, хуримтлагдсан дүнг `SUM(revenue) OVER (ORDER BY revenue DESC)` цонхны функцээр авсан.

SQL код

```sql
CREATE OR REPLACE FUNCTION pct_of(p_part NUMERIC, p_whole NUMERIC)
RETURNS NUMERIC
LANGUAGE sql
IMMUTABLE
AS $$
    -- p_whole нь 0 бол NULLIF нь NULL болгоно; NULL-д хуваавал NULL гарна
    SELECT ROUND(100 * p_part / NULLIF(p_whole, 0), 1);
$$;

DROP TABLE IF EXISTS category_revenue;

-- 1997 оны ангилал бүрийн хямдрал хассан орлого ба борлуулсан тоо ширхэг
CREATE TEMP TABLE category_revenue AS
SELECT
    c.category_name,
    ROUND(SUM((od.unit_price * od.quantity * (1 - od.discount))::numeric), 2) AS revenue,
    SUM(od.quantity) AS units_sold
FROM order_details AS od
JOIN orders AS o ON o.order_id = od.order_id
JOIN products AS p ON p.product_id = od.product_id
JOIN categories AS c ON c.category_id = p.category_id
WHERE EXTRACT(YEAR FROM o.order_date) = 1997
GROUP BY c.category_name;

-- Эзлэх хувь ба хуримтлагдсан хувь. SUM(revenue) OVER () нь нийт орлого.
SELECT
    category_name,
    revenue,
    units_sold,
    pct_of(revenue, SUM(revenue) OVER ()) AS revenue_pct,
    pct_of(SUM(revenue) OVER (ORDER BY revenue DESC), SUM(revenue) OVER ()) AS cum_pct
FROM category_revenue
ORDER BY revenue DESC;

-- Нийт орлогын 50%-ийг бүрдүүлэгч ангиллууд: өмнөх ангиллуудын хуримтлагдсан
-- хувь (cum_pct - revenue_pct) 50-д хүрээгүй бол тухайн ангилал орно.
SELECT category_name, revenue_pct, cum_pct
FROM (
    SELECT
        category_name,
        pct_of(revenue, SUM(revenue) OVER ()) AS revenue_pct,
        pct_of(SUM(revenue) OVER (ORDER BY revenue DESC), SUM(revenue) OVER ()) AS cum_pct
    FROM category_revenue
) AS t
WHERE cum_pct - revenue_pct < 50
ORDER BY cum_pct;

DROP TABLE IF EXISTS category_revenue;
```

Үр дүн

category_name	revenue	units_sold	revenue_pct	cum_pct
Dairy Products	115387.64	4374	18.7	18.7
Beverages	103924.31	3996	16.8	35.5
Confections	82657.75	4137	13.4	48.9
Meat/Poultry	80975.11	2189	13.1	62.1
Seafood	66959.22	3679	10.9	72.9
Grains/Cereals	56871.82	2636	9.2	82.1
Condiments	55368.59	2895	9.0	91.1
Produce	54940.77	1583	8.9	100.0

Зураг 7.1: 1997 оны ангиллын орлого, эзлэх хувь ба хуримтлагдсан хувь (нийт 617,085.21).

category_name	revenue_pct	cum_pct
Dairy Products	18.7	18.7
Beverages	16.8	35.5
Confections	13.4	48.9
Meat/Poultry	13.1	62.1

Зураг 7.2: Нийт орлогын 50%-ийг бүрдүүлэгч ангиллууд.

Нийт орлогын 50%-ийг **Dairy Products, Beverages, Confections, Meat/Poultry** гэсэн 4 ангилал бүрдүүлж байна. Эхний гурав нь хамтдаа 48.9% буюу 50%-д арай хүрэхгүй тул Meat/Poultry нэмэгдэж 62.1% болсноор 50% давж байна. Асуулга нь өмнөх ангиллуудын хуримтлагдсан хувь (`cum_pct - revenue_pct`) 50-д хүрээгүй ангиллуудыг сонгодог.

Даалгавар 8

customer_rfm() — функц доторх түр хүснэгт, сегментчлэл

Ашигласан арга: PL/pgSQL функц доторх `ON COMMIT DROP` түр хүснэгт

RFM шинжилгээ хэд хэдэн алхамтай тул эхний алхмын үр дүнг (recency, frequency, monetary) `rfm_base` түр хүснэгтэд хадгалж, дараагийн алхмуудыг түүн дээр хийсэн. `ON COMMIT DROP` нь дуудлагын transaction дуусахад хүснэгтийг автоматаар устгадаг бол эхний `DROP TABLE IF EXISTS` нь нэг transaction дотор функцийг дахин дуудахад "already exists" алдаа гарахаас хамгаална. Оноог `UPDATE`-ээр хадгалахын оронд эцсийн `SELECT` дотор `NTILE(4)`-ээр шууд тооцсон нь хамгийн энгийн арга: нэмэлт хүснэгт ч, `UPDATE` ч шаардлагагүй.

Бусад шийдвэр:

- Recency бага байх нь сайн тул `NTILE(4) OVER (ORDER BY recency_days DESC)` гэж эрэмбэлсэн: хамгийн удаан захиалаагүй хэсэг 1, хамгийн сүүлд захиалсан хэсэг 4 оноо авна. F, M оноо өсөх эрэмбээр тооцогдоно.
- `customer_id` дээр индекс үүсгэж `ANALYZE` ажиллуулсан. Autovacuum түр хүснэгтэд статистик цуглуулдаггүй тул `ANALYZE`-г гараар хийх шаардлагатай.
- Захиалга хийгээгүй 2 харилцагчийн recency тодорхойгүй тул `JOIN` ашиглаж тэднийг оруулаагүй (89 харилцагч). Захиалга `order_details`-тэй холбогдсон тул frequency-г `COUNT(DISTINCT o.order_id)`-ээр тоолсон.

SQL код

```sql
CREATE OR REPLACE FUNCTION customer_rfm(p_asof DATE DEFAULT DATE '1998-05-06')
RETURNS TABLE (
    customer_id   TEXT,
    company_name  TEXT,
    recency_days  INT,
    frequency     BIGINT,
    monetary      NUMERIC,
    r_score       INT,
    f_score       INT,
    m_score       INT,
    segment       TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Ижил transaction дотор дахин дуудахад алдаа гарахгүй
    DROP TABLE IF EXISTS rfm_base;

    -- 1-р алхам: харилцагч бүрийн R, F, M утга
    CREATE TEMP TABLE rfm_base ON COMMIT DROP AS
    SELECT
        c.customer_id::TEXT AS customer_id,
        c.company_name::TEXT AS company_name,
        p_asof - MAX(o.order_date) AS recency_days,
        COUNT(DISTINCT o.order_id) AS frequency,
        SUM((od.unit_price * od.quantity * (1 - od.discount))::numeric) AS monetary
    FROM customers AS c
    JOIN orders AS o ON o.customer_id = c.customer_id
    JOIN order_details AS od ON od.order_id = o.order_id
    WHERE o.order_date <= p_asof
    GROUP BY c.customer_id, c.company_name;

    -- 2-р алхам: индекс ба статистик
    CREATE INDEX ON rfm_base (customer_id);
    ANALYZE rfm_base;

    -- 3-р алхам: NTILE(4)-ээр оноо өгч, CASE-ээр сегментлэх.
    -- Recency бага байх нь сайн тул DESC эрэмбээр хамгийн шинэ нь 4 оноо авна.
    RETURN QUERY
```

```
    SELECT
        s.customer_id,
        s.company_name,
        s.recency_days,
        s.frequency,
        s.monetary,
        s.r_score,
        s.f_score,
        s.m_score,
        CASE
            WHEN s.r_score >= 3 AND s.f_score >= 3 THEN 'Champion'
            WHEN s.r_score >= 3 THEN 'New/Promising'
            WHEN s.f_score >= 3 THEN 'At risk'
            ELSE 'Hibernating'
        END
    FROM (
        SELECT
            b.*,
            NTILE(4) OVER (ORDER BY b.recency_days DESC) AS r_score,
            NTILE(4) OVER (ORDER BY b.frequency) AS f_score,
            NTILE(4) OVER (ORDER BY b.monetary) AS m_score
        FROM rfm_base AS b
    ) AS s;
END;
$$;

-- (а) Сегмент тус бүрийн харилцагчийн тоо, дундаж monetary
SELECT segment, COUNT(*) AS customers_count, ROUND(AVG(monetary), 2) AS avg_monetary
FROM customer_rfm()
GROUP BY segment
ORDER BY avg_monetary DESC;

-- (б) 'At risk' сегментийн monetary-аар эхний 10 харилцагч
SELECT customer_id, company_name, recency_days, frequency,
       ROUND(monetary, 2) AS monetary, r_score, f_score, m_score
FROM customer_rfm()
WHERE segment = 'At risk'
ORDER BY monetary DESC
LIMIT 10;
```

Үр дүн

segment	customers_count	avg_monetary
Champion	31	28642.19
At risk	13	14701.53
New/Promising	13	5631.26
Hibernating	32	3548.71

Зураг 8.1: (а) Сегмент тус бүрийн харилцагчийн тоо ба дундаж monetary. Champion сегментийн дундаж худалдан авалт Hibernating-ээс 8 дахин их.

customer_id	company_name	recency_days	frequency	monetary	r_score	f_score	m_score
MEREP	Mère Paillarde	188	13	28872.19	1	4	4
FRANK	Frankenversand	27	15	26656.56	2	4	4
BERGS	Berglunds snabbköp	63	18	24927.58	1	4	4
BLONP	Blondesddsl père et fils	114	11	18534.08	1	3	4
SEVES	Seven Seas Imports	91	9	16215.33	1	3	3
VAFFE	Vaffeljernet	34	11	15843.92	2	3	3
AROUT	Around the Horn	26	13	13390.65	2	4	3
SPLIR	Split Rail Beer & Ale	42	9	11441.63	2	3	3
VICTE	Victuailles en stock	103	10	9182.43	1	3	3
MAGAA	Magazzini Alimentari Riuniti	51	10	7176.21	2	3	3

Зураг 8.2: (б) “At risk” сегментийн monetary-аар эхний 10 харилцагч. Жишээ нь Mère Paillarde 13 захиалга, 28,872.19 дүнтэй ч сүүлийн 188 хоногт захиалга хийгээгүй.

Даалгавар 9

restock_products() — өгөгдөл өөрчилдөг функц

Ашигласан арга: өгөгдөл өөрчилдөг PL/pgSQL функц

Шалгах, шинэчлэх, логлох, алдаа боловсруулах алхмуудыг нэг функцэд нэгтгэснээр бүгд нэг transaction дотор атомаар биелнэ: алдаа гарвал бүх өөрчлөлт буцна. `UPDATE ... RETURNING`-ийг `WITH changed AS (...) INSERT`-тэй холбосноор шинэчлэлт ба лог нэг statement-ээр бичигддэг. PostgreSQL 16-д `RETURNING` зөвхөн шинэ утгыг өгдөг тул `products`-ийг өөртэй нь (`FROM products AS old`) холбож хуучин утгыг авсан: `FROM` хэсгийн мөр өөрчлөлтөөс өмнөх төлөвөөр уншигддаг.

Бусад шийдвэр:

- Шинэ утга нь `CEIL(reorder_level * p_factor)::INT`. Нэмэлт `units_on_order < CEIL(...)` нөхцөл нь утгыг зөвхөн нэмэгдүүлнэ. Жишээ нь Condiments ангиллын 66-р бүтээгдэхүүний 100 ширхэг захиалга энэ нөхцөлгүй бол 40 болж буурах байсан.
- Мөрийн тоог `GET DIAGNOSTICS v_count = ROW_COUNT`-оор авч, `RAISE NOTICE`-ээр хэвлээд буцаана.
- Функцийн `EXCEPTION WHEN OTHERS` блок ямар ч алдааг барьж, “restock_products алдаа: ...” гэсэн ойлгомжтой мессежтэйгээр дахин шиднэ.
- Туршилтыг `BEGIN ... ROLLBACK` дотор хийсэн тул `products`-ийн анхны утга сэргэж, скриптийг дахин ажиллуулахад ижил үр дүн гарна.

SQL код

```sql
CREATE OR REPLACE FUNCTION restock_products(
    p_category_name TEXT,
    p_factor        NUMERIC DEFAULT 2.0
) RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_category_id INT;
    v_count       INT;
BEGIN
    IF p_factor IS NULL OR p_factor NOT BETWEEN 1.0 AND 5.0 THEN
        RAISE EXCEPTION 'p_factor нь 1.0-5.0 хооронд байх ёстой (өгсөн утга: %)', p_factor;
    END IF;

    SELECT category_id INTO v_category_id
    FROM categories
    WHERE category_name = p_category_name;

    IF NOT FOUND THEN
        RAISE EXCEPTION '"%" нэртэй ангилал олдсонгүй', p_category_name;
    END IF;

    CREATE TEMP TABLE IF NOT EXISTS restock_log (
        product_id   INT,
        product_name TEXT,
        old_on_order INT,
        new_on_order INT,
        changed_at   TIMESTAMP
    );

    -- UPDATE ... RETURNING-ийн мөрүүдийг шууд логт оруулна.
    -- FROM products AS old нь өөрчлөгдөхөөс өмнөх утгыг өгнө.
    WITH changed AS (
        UPDATE products AS p
        SET units_on_order = CEIL(p.reorder_level * p_factor)::INT
        FROM products AS old
        WHERE old.product_id = p.product_id
          AND p.category_id = v_category_id
```

```
          AND p.discontinued = 0
          AND p.units_in_stock < p.reorder_level
          AND p.units_on_order < CEIL(p.reorder_level * p_factor)  -- зөвхөн нэмэгдүүлнэ
        RETURNING p.product_id, p.product_name, old.units_on_order AS old_on_order,
                  p.units_on_order AS new_on_order
    )
    INSERT INTO restock_log (product_id, product_name, old_on_order, new_on_order, changed_at)
    SELECT product_id, product_name, old_on_order, new_on_order, now()
    FROM changed;

    GET DIAGNOSTICS v_count = ROW_COUNT;

    RAISE NOTICE '%: % бүтээгдэхүүний units_on_order шинэчлэгдлээ (p_factor = %)',
        p_category_name, v_count, p_factor;

    RETURN v_count;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'restock_products алдаа: %', SQLERRM;
END;
$$;

-- Туршилтыг transaction дотор хийж, ROLLBACK-ээр products-ийн анхны утгыг сэргээнэ.
BEGIN;

-- 1) Амжилттай дуудлага
SELECT restock_products('Beverages', 2.0) AS updated_rows;

-- 2) p_factor = 0.5 -> алдаа
DO $$
BEGIN
    PERFORM restock_products('Beverages', 0.5);
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Алдаа: %', SQLERRM;
END;
$$;

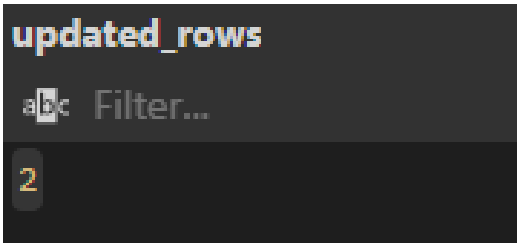
-- 3) Байхгүй ангилал -> алдаа
DO $$
BEGIN
    PERFORM restock_products('Electronics');
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Алдаа: %', SQLERRM;
END;
$$;

-- 4) Логийн үр дүн
SELECT * FROM restock_log ORDER BY product_id;

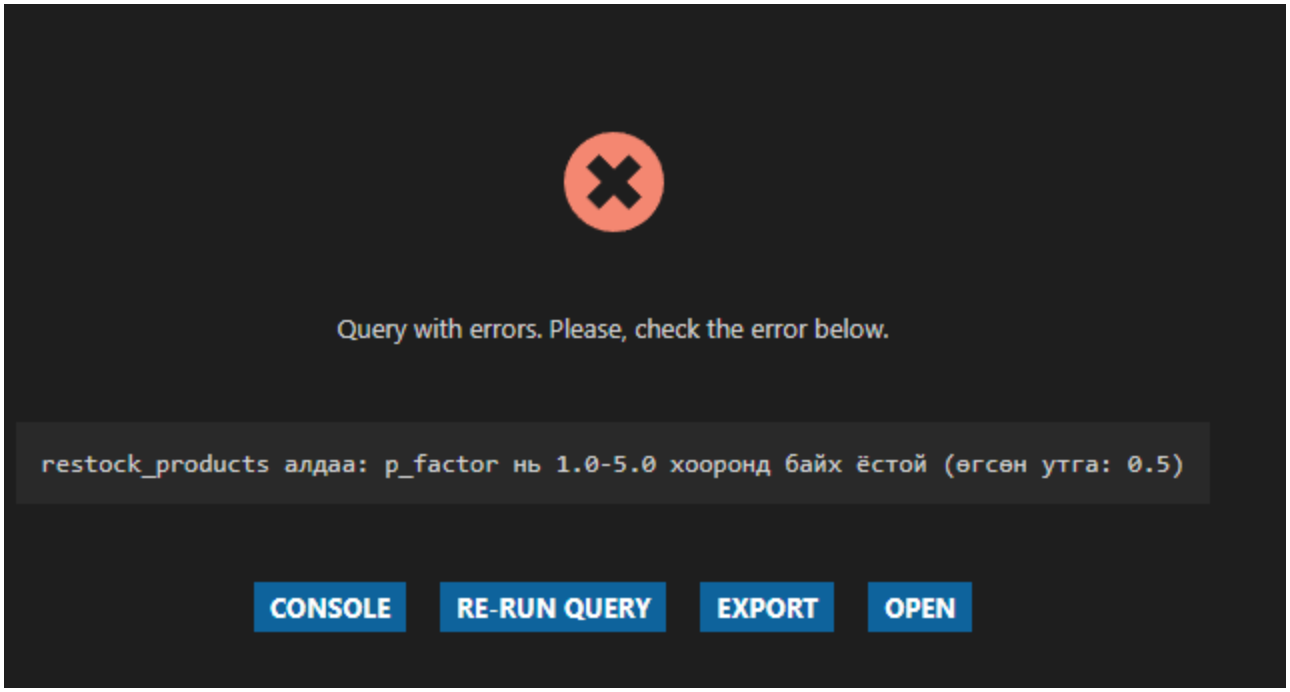
ROLLBACK;

DROP TABLE IF EXISTS restock_log;
```

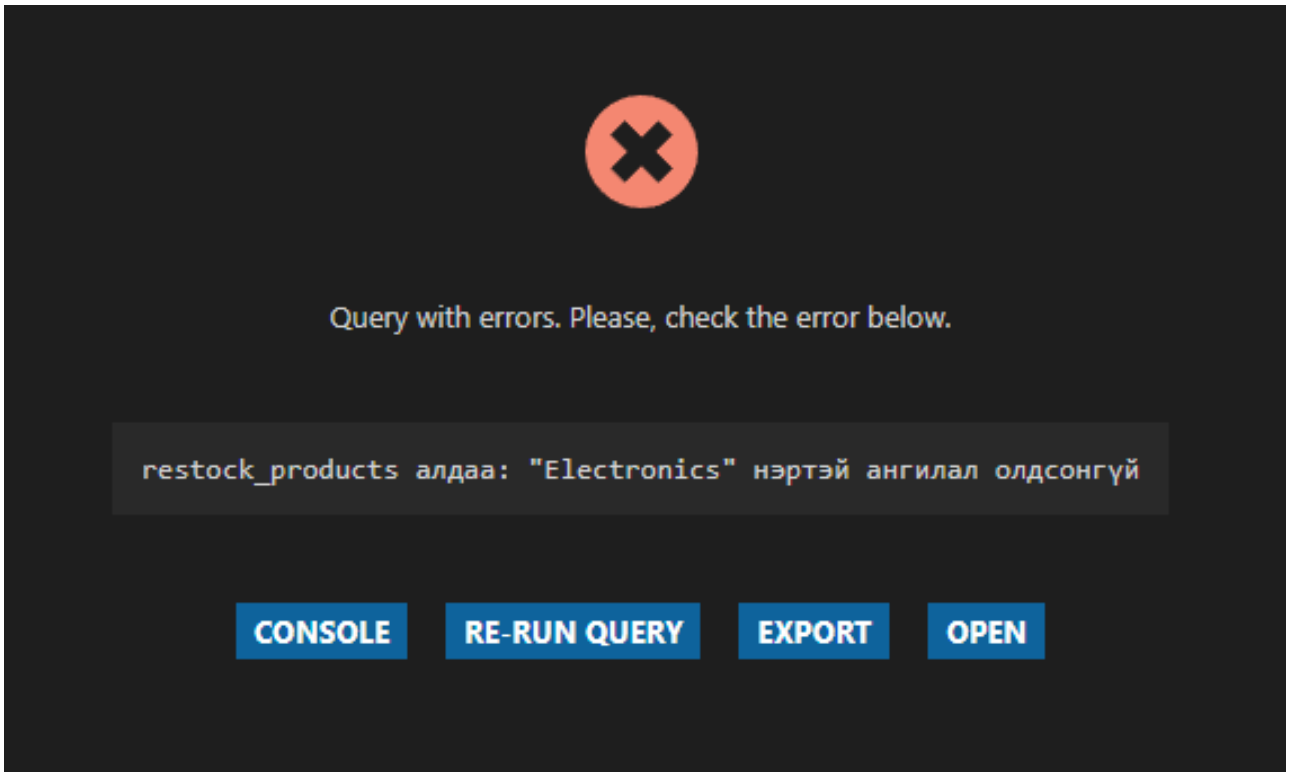
Үр дүн



Зураг 9.1: Амжилттай дуудлага. Beverages ангиллын 2 бүтээгдэхүүн шинэчлэгдсэн. Зогсоосон Chang (`discontinued = 1`) хамрагдаагүй.



Зураг 9.2: `SELECT restock_products('Beverages', 0.5)` — `p_factor = 0.5` үед гарах алдаа.



Зураг 9.3: `SELECT restock_products('Electronics')` — байхгүй ангилал өгөхөд гарах алдаа.

product_id	product_name	old_on_order	new_on_order	changed_at
43	Ipoh Coffee	10	50	2026-10-02 22:10:19.617607
70	Outback Lager	10	60	2026-10-02 22:10:19.617607

Зураг 9.4: Дуудлагын дараах `restock_log`: Ipoh Coffee 10 → 50, Outback Lager 10 → 60 (`reorder_level` × 2).

SQL Server-ийн UDF-д яагаад боломжгүй вэ?

SQL Server функцийг `SELECT` дотор мөр бүрт хэдэн ч удаа, ямар ч дарааллаар дуудаж болох “цэвэр” илэрхийлэл гэж үздэг тул UDF-д гаж нөлөө (side effect) хориотой. UDF дотор байнгын хүснэгтэд `INSERT/UPDATE/DELETE` хийх, `#temp` хүснэгт үүсгэх, `TRY...CATCH`, `THROW/RAISERROR`, динамик SQL (`EXEC`) болон transaction удирдах боломжгүй. Зөвхөн функцийн локал `@table` хувьсагчийг өөрчилж болно. Харин PostgreSQL-д `VOLATILE` функц өгөгдөл өөрчилж, түр хүснэгт үүсгэж, `EXCEPTION` блок ашиглаж чадна.

Функц уу, процедур уу?

Миний бодлоор энэ үйлдэлд `PROCEDURE` (`CALL restock_products(...)`) илүү тохиромжтой. Энэ нь утга тооцдог бус өгөгдөл өөрчилдөг, ганцаараа дуудагддаг үйлдэл бөгөөд процедур нь `SELECT` дотор санамсаргүй дуудагдахгүй, дотроо `COMMIT/ROLLBACK` хийж transaction-ийг удирдаж чадна. Шинэчилсэн мөрийн тоог `INOUT` параметрээр буцааж болно. Харин үр дүнг `SELECT`-д шууд ашиглах, бусад асуулгатай нийлүүлэх шаардлагатай бол функц давуу.

Даалгавар 10

abc_analysis() — Парето шинжилгээ ба гүйцэтгэлийн харьцуулалт

Ашигласан арга: түр хүснэгттэй функц ба зөвхөн CTE-тэй функц

`abc_analysis()` нь тухайн жилийн бүтээгдэхүүн бүрийн орлогыг `product_revenue` түр хүснэгтэд (`ON COMMIT DROP`) хадгалаад, эзлэх хувь, хуримтлагдсан хувь (`ROWS UNBOUNDED PRECEDING`) болон A/B/C ангиллыг тооцно. 7-р даалгаврын `pct_of()`-ийг дахин ашигласан. `abc_analysis_cte()` нь ижил логикийг түр хүснэгтгүй, хоёр CTE-тэй нэг `LANGUAGE sql` асуулгаар бичсэн хувилбар. Хоёр хувилбарын үр дүнг `EXCEPT`-ээр харьцуулахад 0 мөр гарсан тул үр дүн нь ижил.

SQL код

```sql
-- 1-р хувилбар: функц доторх түр хүснэгт
CREATE OR REPLACE FUNCTION abc_analysis(p_year INT)
RETURNS TABLE (
    product_id     INT,
    product_name   TEXT,
    category_name  TEXT,
    revenue        NUMERIC,
    revenue_pct    NUMERIC,
    cum_pct        NUMERIC,
    abc_class      CHAR(1)
)
LANGUAGE plpgsql
AS $$
BEGIN
    DROP TABLE IF EXISTS product_revenue;

    -- Тухайн жилийн бүтээгдэхүүн бүрийн орлого
    CREATE TEMP TABLE product_revenue ON COMMIT DROP AS
    SELECT
        p.product_id::INT AS product_id,
        p.product_name::TEXT AS product_name,
        c.category_name::TEXT AS category_name,
        SUM((od.unit_price * od.quantity * (1 - od.discount))::numeric) AS revenue
    FROM order_details AS od
    JOIN orders AS o ON o.order_id = od.order_id
    JOIN products AS p ON p.product_id = od.product_id
    JOIN categories AS c ON c.category_id = p.category_id
    WHERE EXTRACT(YEAR FROM o.order_date) = p_year
    GROUP BY p.product_id, p.product_name, c.category_name;

    -- Эзлэх хувь, хуримтлагдсан хувь, A/B/C ангилал
    RETURN QUERY
    SELECT
        t.product_id,
        t.product_name,
        t.category_name,
        ROUND(t.revenue, 2),
        t.revenue_pct,
        t.cum_pct,
        (CASE
            WHEN t.cum_pct <= 80 THEN 'A'
            WHEN t.cum_pct <= 95 THEN 'B'
            ELSE 'C'
        END)::CHAR(1)
    FROM (
        SELECT
            r.*,
            pct_of(r.revenue, SUM(r.revenue) OVER ()) AS revenue_pct,
            pct_of(SUM(r.revenue) OVER (ORDER BY r.revenue DESC ROWS UNBOUNDED PRECEDING),
                   SUM(r.revenue) OVER ()) AS cum_pct
        FROM product_revenue AS r
    ) AS t
    ORDER BY t.revenue DESC;
END;
```

```
$$;

-- (а) A/B/C бүлэг тус бүрийн бүтээгдэхүүний тоо, нийт ба дундаж орлого
SELECT abc_class,
       COUNT(*) AS products_count,
       SUM(revenue) AS total_revenue,
       ROUND(AVG(revenue), 2) AS avg_revenue
FROM abc_analysis(1997)
GROUP BY abc_class
ORDER BY abc_class;

-- (б) 'A' бүлгийн бүтээгдэхүүнүүд ангиллаар. Эхний мөр нь хамгийн чухал ангилал.
SELECT category_name,
       COUNT(*) AS a_products,
       SUM(revenue) AS a_revenue
FROM abc_analysis(1997)
WHERE abc_class = 'A'
GROUP BY category_name
ORDER BY a_products DESC, a_revenue DESC;

-- 2-р хувилбар: ижил логик, зөвхөн CTE ашигласан (түр хүснэгтгүй)
CREATE OR REPLACE FUNCTION abc_analysis_cte(p_year INT)
RETURNS TABLE (
    product_id     INT,
    product_name   TEXT,
    category_name  TEXT,
    revenue        NUMERIC,
    revenue_pct    NUMERIC,
    cum_pct        NUMERIC,
    abc_class      CHAR(1)
)
LANGUAGE sql
STABLE
AS $$
    WITH product_revenue AS (
        SELECT
            p.product_id::INT AS product_id,
            p.product_name::TEXT AS product_name,
            c.category_name::TEXT AS category_name,
            SUM((od.unit_price * od.quantity * (1 - od.discount))::numeric) AS revenue
        FROM order_details AS od
        JOIN orders AS o ON o.order_id = od.order_id
        JOIN products AS p ON p.product_id = od.product_id
        JOIN categories AS c ON c.category_id = p.category_id
        WHERE EXTRACT(YEAR FROM o.order_date) = p_year
        GROUP BY p.product_id, p.product_name, c.category_name
    ),
    ranked AS (
        SELECT
            r.*,
            pct_of(r.revenue, SUM(r.revenue) OVER ()) AS revenue_pct,
            pct_of(SUM(r.revenue) OVER (ORDER BY r.revenue DESC ROWS UNBOUNDED PRECEDING),
                   SUM(r.revenue) OVER ()) AS cum_pct
        FROM product_revenue AS r
    )
    SELECT
        product_id,
        product_name,
        category_name,
        ROUND(revenue, 2),
        revenue_pct,
        cum_pct,
        (CASE
            WHEN cum_pct <= 80 THEN 'A'
            WHEN cum_pct <= 95 THEN 'B'
            ELSE 'C'
        END)::CHAR(1)
    FROM ranked
    ORDER BY revenue DESC;
$$;
```

```sql
-- Хоёр хувилбар ижил үр дүн өгч буйг шалгах (0 мөр гарвал ижил)
SELECT * FROM abc_analysis(1997)
EXCEPT
SELECT * FROM abc_analysis_cte(1997);

-- Гүйцэтгэлийн харьцуулалт: түр хүснэгттэй хувилбар
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM abc_analysis(1997);

-- Гүйцэтгэлийн харьцуулалт: CTE хувилбар
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM abc_analysis_cte(1997);
```

Үр дүн

abc_class	products_count	total_revenue	avg_revenue
A	36	492739.40	13687.21
B	21	91004.14	4333.53
C	20	33341.63	1667.08

Зураг 10.1: (а) A/B/C бүлгүүд. 77 бүтээгдэхүүний 36 нь (47%) 1997 оны орлогын 80%-ийг бүрдүүлж байна.

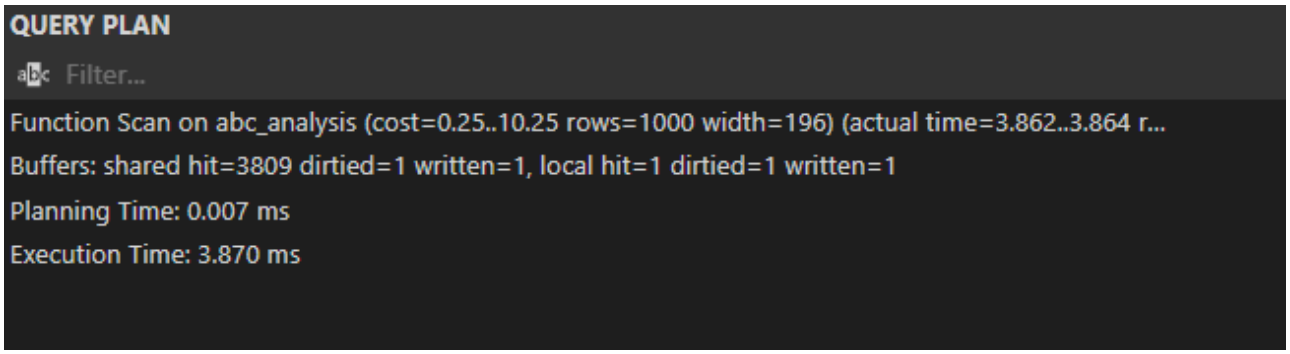
category_name	a_products	a_revenue
Dairy Products	8	111932.84
Meat/Poultry	5	77790.82
Confections	5	59033.10
Beverages	4	74685.64
Produce	4	53940.27
Seafood	4	42594.38
Condiments	4	32101.87
Grains/Cereals	2	40660.48

Зураг 10.2: (б) A бүлгийн бүтээгдэхүүнүүд ангиллаар. **Dairy Products** нь A бүлгийн 8 бүтээгдэхүүн, 111,932.84 орлоготой хамгийн чухал ангилал.

product_id	product_name	category_name	revenue	revenue_pct	cum_pct	abc_class
No data						

Зураг 10.3: Хоёр хувилбарын үр дүнгийн ялгаа 0 мөр.

EXPLAIN (ANALYZE, BUFFERS) харьцуулалт



Зураг 10.4: Түр хүснэгттэй хувилбарын төлөвлөгөө. PL/pgSQL функц ганц `Function Scan` зангилаа болж харагдана.



Зураг 10.5: CTE хувилбарын төлөвлөгөө (SQLTools). Урт мөрүүд тасарч, төлөвлөгөөний эхний хэсэг харагдана.

Хугацааг тогтвортой болгохын тулд нэг сешнд хувилбар бүрийг ээлжлэн 12 удаа ажиллуулсан. Эхний дуудлагыг тусад нь, үлдсэн 11-ийг медианаар харуулав. Түр хүснэгттэй функцийн дотоод алхмуудын хугацааг `auto_explain` (`log_nested_statements`) модулиар хэмжсэн.

Үзүүлэлт	Түр хүснэгттэй (`abc_analysis`)	Зөвхөн CTE (`abc_analysis_cte`)
Execution Time, эхний дуудлага	10.59 ms (PL/pgSQL эмхэтгэл орсон)	2.52 ms
Execution Time, медиан (11 удаа)	3.78 ms (3.75–5.02)	2.51 ms (2.47–2.57)
Planning Time, медиан	0.01 ms	0.36 ms
Нийт (planning + execution)	≈ 3.79 ms	≈ 2.87 ms
Buffers	shared hit ≈ 3650, shared dirtied/written 1, local hit/dirtied/written 1	shared hit 3361, бичилт байхгүй
EXPLAIN-д харагдах төлөвлөгөө	Ганц `Function Scan`, мөрийн тоог 1000 гэж таамагласан (бодит 77)	Бүрэн төлөвлөгөө: orders `Seq Scan` → order_details `Bitmap Heap Scan` → products `Index Scan` → categories `Hash Join` → `GroupAggregate` → `Sort` → 2 × `WindowAgg`
Дотоод алхмууд	`CREATE TEMP TABLE AS` ≈ 3.4 ms, эцсийн `SELECT` ≈ 0.13 ms (статистикгүй түр хүснэгтэд 630 мөр гэж таамагласан, бодит 77)	Нэг асуулга. Функц inline хийгдэж, гадна асуулгатай хамт төлөвлөгдсөн

Хоёр хувилбарын холболт (join), нэгтгэлийн төлөвлөгөө адилхан. Ялгаа нь түр хүснэгттэй хувилбар хүснэгт үүсгэх, системийн каталогт бичих, устгах нэмэлт ажил хийдэгт оршино. SQLTools урт мөрийг таслан харуулдаг тул CTE хувилбарын бүрэн төлөвлөгөөг Зураг 10.6-д `psql`-ээр үзүүлэв.

Дүгнэлт

Энэ тохиолдолд CTE хувилбар илүү тохиромжтой. Өгөгдөл бага (77 бүтээгдэхүүн, 1059 захиалгын мөр) бөгөөд завсрын үр дүнг ганц удаа ашигладаг тул түр хүснэгт үүсгэж, каталогт бичиж, устгах зардал (ойролцоогоор 1 ms) нь ашгаасаа их байна. Хэмжилтээр CTE хувилбар ойролцоогоор 2.9 ms, түр хүснэгттэй хувилбар 3.8 ms, эхний дуудлагад PL/pgSQL эмхэтгэл нэмэгдэж 10.6 ms болсон. Мөн `LANGUAGE sql` функц асуулга руу inline хийгддэг тул planner бүх төлөвлөгөөг харж оновчилдог бол PL/pgSQL функц planner-ийн хувьд “хар хайрцаг” (`Function Scan`, 1000 мөр гэсэн таамаг) болдог. Харин завсрын үр дүн том бөгөөд олон асуулгад дахин ашиглагддаг, түүн дээр индекс үүсгэх шаардлагатай, эсвэл CTE-ийн мөрийн тооцоолол буруу байж төлөвлөгөө муу гардаг үед `ANALYZE` хийсэн түр хүснэгт planner-д бодит статистик өгч илүү хурдан ажиллана. Мөн алхам бүрийн үр дүнг шалгах шаардлагатай, олон алхамтай урт боловсруулалтад түр хүснэгт илүү ойлгомжтой.

```
psql · dedibeat@northwind · PostgreSQL 16

northwind=> EXPLAIN (ANALYZE, BUFFERS)
northwind-> SELECT * FROM abc_analysis_cte(1997);
                                                
```